COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/01/2026	**DATE SUBMITTED:**	08/01/2026
NAME:	Cabug, Jeffordniel H.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON NO. 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A:

[*] main.cpp [*] CABUG_Seatwork4_1.cpp

```cpp
#include <iostream>
#include <stack> // call the stack from the STL ILO A

int main (){

    std::stack<int> newStack; // created an object

    // testinf the empty() function to check if the stack is empty
    std::cout<<"Is the stack empty? "<< newStack.empty()<<std::endl;

    //insert a 3 elements:
    newStack.push(3);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;

    std::cout<<"Is the stack empty? "<< newStack.empty()<<std::endl;

    // emplace -> construct and insert element -> public member function
    std::cout<<"====================EMPLACE===================="<<std::endl;
    newStack.emplace(9);
    std::cout<<"Top of the stack: "<<newStack.top()<<std::endl;
    std::cout<<"Size of the stack: "<<newStack.size()<<std::endl;

    newStack.push(6);
    newStack.push(7);
    newStack.push(8);
    std::cout<<"Size of the stack is: "<< newStack.size()<<std::endl;

    // swap?? -? swap the content of the public member functions
    std::stack<int> swapStack;
    swapStack.push(10);
    swapStack.push(11);
    swapStack.push(12);
    swapStack.push(13);
    swapStack.push(14);

    newStack.swap(swapStack);
    std::cout<<"====================SWAPPED===================="<<std::endl;
    std::cout<<"newStack swapped top: "<<newStack.top()<<std::endl;
    std::cout<<"swapStack new top: "<<swapStack.top()<<std::endl;

    return 0;
}
```

OUTPUT:

```
Is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
Is the stack empty? 0
====================EMPLACE====================
Top of the stack: 9
Size of the stack: 4
Size of the stack is: 7
====================SWAPPED====================
newStack swapped top: 14
swapStack new top: 8

--------------------------------
Process exited after 0.008363 seconds with return value 0
Press any key to continue . . .
```

ILO A CODE ANALYSIS:

The program includes stack. First, it checks if the stack is empty using the empty() function. Since the stack has no elements at the beginning, it returns 1 (TRUE). The program then adds values to the stack and displays the top element using the top() function while removing elements with pop(), showing the Last In, First Out (LIFO) behavior of a stack. After that, it checks again if the stack is empty, and it returns 0 (FALSE) because there are still elements inside. The program also uses the emplace() function to insert a new element, size() to display the number of elements, and swap() to exchange the contents of two stacks.

ILO B:

main.cpp [*] CABUG_Seatwork4_1.cpp

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

//prototyping
bool isFull();
bool isEmpty();
void push();
void pop();
void Top();

//display all
void displayAll();

int main(){
    std::cout<<"Enter the number of maximum elements for the new stack (max: 100): ";
    std::cin>>elements;

    while(true){
        int choice = 0;
        std::cout<<"Select Stack Operation:"<<std::endl;
        std::cout<<"1. PUSH\n2. POP\n3. VIEW TOP\n4. IS EMPTY?\n5. ISFULL\n6. displayAll"<<std::endl;
        std::cin >> choice;

        switch(choice){
            case 1:
                push();
                break;

            case 2:
                pop();
                break;

            case 3:
                Top();
                break;

            case 4:
                std::cout<<isEmpty()<<std::endl;
                break;

            case 5:
                std::cout<<isFull()<<std::endl;
                break;

            case 6:
                displayAll();
                break;

            default:
                std::cout<<"Invalid Choice"<<std::endl;
                break;
        }
    }

```

main.cpp [*] CABUG_Seatwork4_1.cpp

```
58      return 0;
59  }
60
61  bool isEmpty(){
62      //top is -1
63      if(top == -1){
64          std::cout<<"Stack Underflow"<<std::endl;
65          return true;
66      }
67      return false;
68  }
69
70  bool isFull(){
71      //not equal to maxCap
72      if(top >= elements){
73          std::cout<<"Stack Overflow"<<std::endl;
74          return true;
75      }
76      return false;
77
78  }
79
80  void push(){
81      //check if stack is full, if yes, return error
82      if (isFull()){
83          return;
84      }
85      //ask user input values to be pushed into stack
86      std::cout<<"What number would you like to add? ";
87      std::cin>>newData;
88      //store inside stack
89      stack[++top] = newData;
90  }
91
92  void pop(){
93      // check if empty
94      if (isEmpty()){
95          return;
96      }
97
98      //display element to be popped
99      std::cout<<"Popping: "<<stack[top]<<std::endl;
100
101     //decrement top:
102     top--;
103 }
104
105 void Top(){
106     //error checking
107     if(isEmpty()){
108         return;
109     }
110     //display top
111     std::cout<<"The top element is"<<stack[top]<<std::endl;
112
113 }
114
```

```
115 void displayAll(){
116     if (isEmpty()){
117         return;
118     }
119     for (int i = top; i >=0; i--){
120         std::cout<< stack[i]<<" ";
121 
122     }
123     std::cout <<std::endl;
124 }
```

OUTPUT:

1.) **PUSH**

```
Enter the number of maximum elements for the new stack (max: 100): 5
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
1
What number would you like to add? 5
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
1
What number would you like to add? 4
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
1
What number would you like to add? 3
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

2.) DISPLAYALL

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
6
3 4 5
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

3.) TOP

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
3
The top element is3
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

4.) POP

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
2
Popping: 3
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

DISPLAY AFTER POPPING

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
6
4 5
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

5.) ISEMPTY

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
4
0
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

6.) ISFULL

```
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
5
0
Select Stack Operation:
1. PUSH
2. POP
3. VIEW TOP
4. IS EMPTY?
5. ISFULL
6. displayAll
```

ILO B CODE ANALYSIS:

The stack is stored in an array with a maximum capacity of 100 elements, while the user can specify the actual stack size at the beginning of the program. The variable top is initialized to -1 to indicate that the stack is empty. Before every push or pop operation, the program checks for stack overflow and underflow using the isFull() and isEmpty() functions to prevent invalid operations. Each function has a specific purpose, push() adds a new element to the top of the stack. pop() removes the top element from the stack. Top() displays the current top element. displayAll() prints all stack elements from top to bottom. isEmpty() and isFull() verify whether the stack can perform insertion or deletion. The menu-driven loop allows the user to repeatedly perform operations until the program is manually terminated.

ILO B2:

HEADER:

listStack.h

listStack.h > displayAll<T>()

```
#ifndef LISTSTACK_H
#define LISTSTACK_H

//class node with data and next numbers
template <typename T>
class Node{
    public:
        T data;
        Node<T>* next = nullptr;
};

//create a pointer/ reference tip that always points to current top node

// global
template <typename T>
Node<T>* head = nullptr;


//pushing operation:
template <typename T>
void push(T newData){
    //create a new node
    Node<T>* newNode = new Node<T>;
    // assign the data to the new node
    newNode->data = newData;
    //point the newNode next to the current top
    newNode->next = head<T>;
    // asign the new Top to the new Node
    head<T> = newNode;
}

//isEmpty()
```

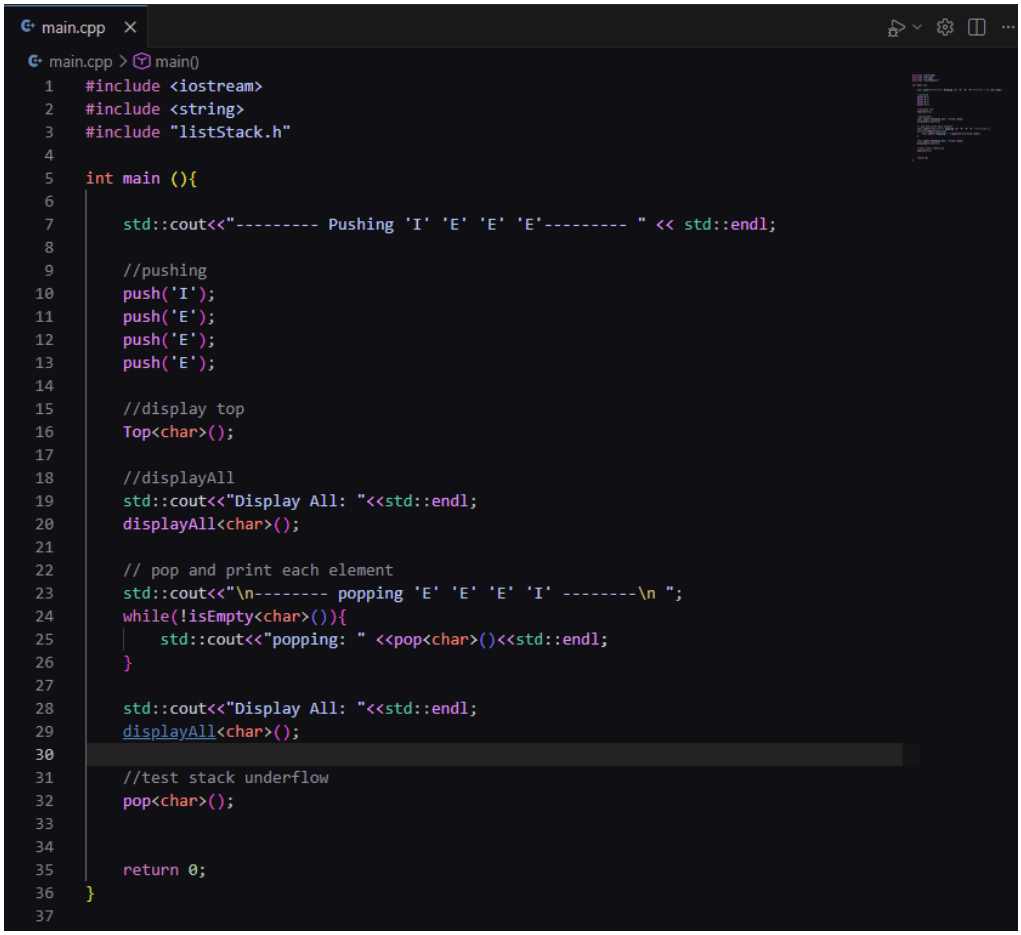
listStack.h

listStack.h > displayAll<T>()

```
1   #ifndef LISTSTACK_H
33  template <typename T>
34  bool isEmpty(){
35      return head<T> == nullptr;
36  }
37
38  template <typename T>
39  T pop(){
40      //if the stack is empty error cathc:
41      if(isEmpty<T>()){
42          std::cout<<"Stack underflow"<<std::endl;
43      }
44
45      // saving a current top node to a temporary variable
46      Node<T>* temp = head<T>;
47      // store the data to a temporary variable
48      T tempVal = temp->data;
49      // move the head to next node
50      head<T> = head<T>-> next;
51      //free up the memory
52      delete temp;
53      //return the temporary variable data
54      return tempVal;
55  }
56
57  template <typename T>
58  void Top(){
59      //what if the stack is empty?
60      if(isEmpty<T>()) return;
61      std::cout<<"the top is: " << head<T>->data<<std::endl;
62  }
63
```

```
63
64  template <typename T>
65  void displayAll(){
66
67      Node<T>* temp = head<T>;
68
69      if(isEmpty<T>()){
70          std::cout << "List is empty" << std::endl;
71          return;
72      }
73
74      while(head<T> != nullptr){
75          std::cout<< head<T> ->data;
76          if(head<T> ->next != nullptr){
77              std::cout<<"->";
78          }
79          head<T>  = head<T>->next;
80      }
81      head<T> = temp;
82  }
83
84  #endif
85
```

MAIN.CPP:



```cpp
#include <iostream>
#include <string>
#include "listStack.h"

int main (){

    std::cout<<"--------- Pushing 'I' 'E' 'E' 'E'--------- " << std::endl;

    //pushing
    push('I');
    push('E');
    push('E');
    push('E');

    //display top
    Top<char>();

    //displayAll
    std::cout<<"Display All: "<<std::endl;
    displayAll<char>();

    // pop and print each element
    std::cout<<"\n-------- popping 'E' 'E' 'E' 'I' --------\n ";
    while(!isEmpty<char>()){
        std::cout<<"popping: " <<pop<char>()<<std::endl;
    }

    std::cout<<"Display All: "<<std::endl;
    displayAll<char>();

    //test stack underflow
    pop<char>();


    return 0;
}
```

OUTPUT:

```
[Running] cd "c:\Users\TIPQC\Documents\CPE21S3_CABUG\IL
--------- Pushing 'I' 'E' 'E' 'E'---------
the top is: E
Display All:
E->E->E->I
-------- popping 'E' 'E' 'E' 'I' --------
 popping: E
popping: E
popping: E
popping: I
Display All:
List is empty
Stack underflow

[Done] exited with code=3221225477 in 0.471 seconds
```

OBSERVATION:
The program has a header file that contains a list stack that pushes, displays, pop strings and checks the list if it's empty and it will print out "stack underflow".

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C:

A. Stacks Using Array

```cpp
#include <iostream>
#include <string>
using namespace std;

class Stack {
private:
    char arr[100];
    int top;

public:
    Stack() {
        top = -1;
    }

    bool isEmpty() {
        return top == -1;
    }

    bool isFull() {
        return top == 99;
    }

    void push(char x) {
        if (!isFull()) {
            arr[++top] = x;
        }
    }

    char pop() {
        if (!isEmpty()) {
            return arr[top--];
        }
        return '\0';
    }
};

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalanced(string expr) {
    Stack s;

    for (char ch : expr) {

        if (ch == '(' || ch == '{' || ch == '[') {
            s.push(ch);
        }

        else if (ch == ')' || ch == '}' || ch == ']') {

```

```
54              if (s.isEmpty()) {
55                  return false;
56              }
57
58              char open = s.pop();
59
60              if (!isMatching(open, ch)) {
61                  return false;
62              }
63          }
64      }
65
66      return s.isEmpty();
67  }
68
69  int main() {
70
71      string expression;
72
73      cout << "Enter Expression: ";
74      getline(cin, expression);
75
76      if (checkBalanced(expression))
77          cout << "Expression is Balanced." << endl;
78      else
79          cout << "Expression is NOT Balanced." << endl;
80
81      return 0;
82  }
```

B. Stacks Using Linked List

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Node {
    char data;
    Node* next;
};

class Stack {
private:
    Node* top;

public:
    Stack() {
        top = nullptr;
    }

    bool isEmpty() {
        return top == nullptr;
    }

    void push(char x) {
        Node* newNode = new Node;
        newNode->data = x;
        newNode->next = top;
        top = newNode;
    }

    char pop() {

        if (isEmpty())
            return '\0';

        Node* temp = top;
        char value = temp->data;

        top = top->next;
        delete temp;

        return value;
    }
};

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalanced(string expr) {

    Stack s;
```

```
54
55      for (char ch : expr) {
56
57          if (ch == '(' || ch == '{' || ch == '[') {
58              s.push(ch);
59          }
60
61          else if (ch == ')' || ch == '}' || ch == ']') {
62
63              if (s.isEmpty())
64                  return false;
65
66              char open = s.pop();
67
68              if (!isMatching(open, ch))
69                  return false;
70          }
71      }
72
73      return s.isEmpty();
74  }
75
76  int main() {
77
78      string expression;
79
80      cout << "Enter Expression: ";
81      getline(cin, expression);
82
83      if (checkBalanced(expression))
84          cout << "Expression is Balanced." << endl;
85      else
86          cout << "Expression is NOT Balanced." << endl;
87
88      return 0;
89  }
```

C. Stacks Using STL

```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isMatching(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '{' && close == '}') ||
           (open == '[' && close == ']');
}

bool checkBalanced(string expr) {

    stack<char> s;

    for (char ch : expr) {

        if (ch == '(' || ch == '{' || ch == '[') {
            s.push(ch);
        }

        else if (ch == ')' || ch == '}' || ch == ']') {

            if (s.empty())
                return false;

            char open = s.top();
            s.pop();

            if (!isMatching(open, ch))
                return false;
        }
    }

    return s.empty();
}

int main() {

    string expression;

    cout << "Enter Expression: ";
    getline(cin, expression);

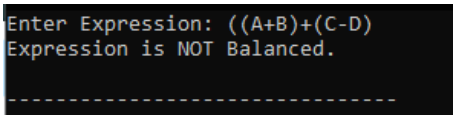
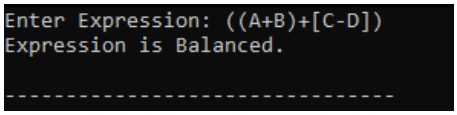
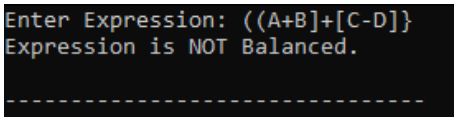
    if (checkBalanced(expression))
        cout << "Expression is Balanced." << endl;
    else
        cout << "Expression is NOT Balanced." << endl;

    return 0;
}
```

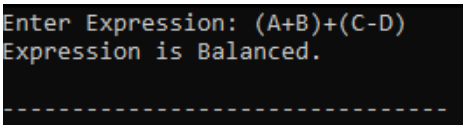
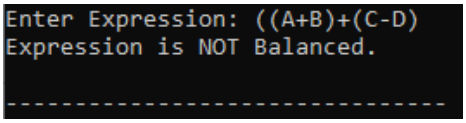
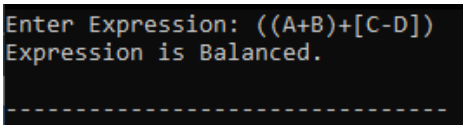
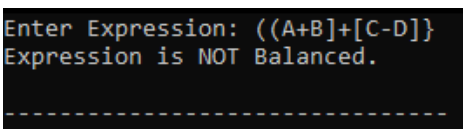
OUTPUT:

STACKS USING ARRAY

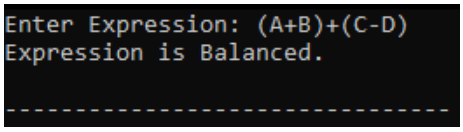
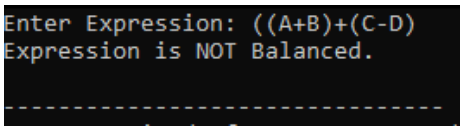
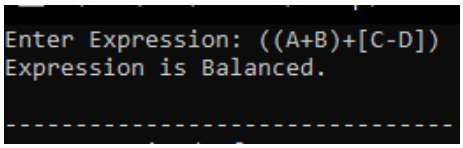
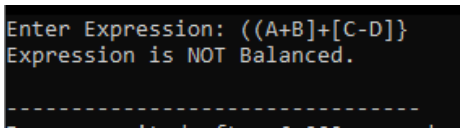
Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	Enter Expression: (A+B)+(C-D) Expression is Balanced. --------------------------------	All opening symbols have matching closing symbols.

((A+B)+(C-D)	N		One opening parenthesis (has no matching closing parenthesis.
((A+B)+[C-D])	Y		Parentheses () and brackets [] are properly matched and correctly nested.
((A+B]+[C-D]}	N		The symbols] and } do not match the expected opening symbols, making the expression unbalanced.

STACKS USING LINKED LIST

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The linked list stack checked every symbol and found no errors. The expression is balanced.
((A+B)+(C-D)	N		There is still one opening parenthesis left in the linked list stack. The expression is not balanced.
((A+B)+[C-D])	Y		The linked list stack matched all the symbols in the correct order. The expression is balanced.
((A+B]+[C-D]}	N		The linked list stack found the wrong closing symbols. It is not balanced

STACKS USING STL

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The STL stack checked the symbols and found that they all match. The expression is balanced.
((A+B)+(C-D)	N		The STL stack still has one opening parenthesis after checking the expression. It is not balanced
((A+B)+[C-D])	Y		The STL stack matched all the parentheses and brackets correctly. The expression is balanced.
((A+B]+[C-D]}	N		The STL stack found mismatched symbols, so the expression is not balanced.

OBSERVATION:

The program uses Array Stack, Linked List Stack, and STL Stack all gave the same results when checking the expressions. The valid expressions were accepted because all opening symbols had the correct closing symbols. The invalid expressions were rejected because they had missing or incorrect symbols. The only difference between the three implementations is how the stack is made. The array stack uses a fixed-size array, the linked list stack uses nodes, and the STL stack uses the built-in stack in C++. Each of them gave the same results.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt Ai overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts and grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I use ai for concepts of stacks and for STL also I use ai for grammar correction.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.